

EP2C: Explainable Paper-to-Code

Vraj Patel

University of Maryland, College Park
College Park, USA
vpatel08@terpmail.umd.edu

Ayhan Mehdiyev

University of Maryland, College Park
College Park, USA
aymehdi1@terpmail.umd.edu

Colin McElroy

University of Maryland, College Park
College Park, USA
cmcelroy@terpmail.umd.edu

Saleem Salavudheen

University of Maryland, College Park
College Park, USA
saleems@terpmail.umd.edu

Abstract

A lot of machine learning papers do not include complete or easy-to-follow code, which makes reproducing results difficult. Existing paper-to-code systems can generate code but they often do not explain how the code relates to the original paper or why certain choices were made. We built EP2C (Explainable Paper-to-Code) a system that converts research papers into code repositories while also providing clear explanations and traceability back to the paper. EP2C parses papers, plans the repository structure, generates code in stages and adds an explanation layer that links code components to specific sections of the paper and highlights missing information such as datasets or hyperparameters. The code is available at: <https://github.com/vupatel08/473-Capstone-Project-EP2C>.

1 Introduction

Each year thousands of research papers are published most of which lack full code or at the very least, a clear guide on how to recreate their experiments. This can be quite challenging for students, developers, and even other researchers who want to validate results, build upon previous work, or even understand how things were actually made. Thus, scientific advancement stalls, not due to a lack of ideas, but rather because the concepts are too complex to implement.

Through the use of large language models (LLMs), recent systems, such as AutoP2C and Paper2Code, have been developed in order to lessen this disparity. They take in research papers, parse their text and figures, then try to generate runnable code from them. It is certainly a promising step forward. That said, there is no actual reason to believe that these tools implement the methods described in the paper. Seldom do these tools explain why something was done in a certain way, ignore gaps, and typically result in users being left with code that feels disjointed from the original research.

That is the exact problem that EP2C (Explainable Paper-to-Code) was designed to resolve. EP2C does more than simply generate code. It demystifies the process, adding that essential transparency and confidence to what is being produced. At its core, EP2C integrates structured analysis tools such as MinerU with LLMs to mine architectural and methodological knowledge from papers. It then develops modular code and provides a link to the original via each part of the paper. If any key ingredients are missing, like datasets or hyperparameters, EP2C will flag these explicitly, preventing anything from being lost in translation.

One of the strengths of EP2C is its user interface. It allows users to see the research paper and the code it produces, next to each other. These semantic cues and structured highlights reveal how different parts of the paper are linked to code, making it easier to trace out logic, apprehend design choices where they exist, and debug when necessary. Rather than simply determining whether the code executes, EP2C aims to help users comprehend why and how it works. That transition, from execution to explanation, is also what makes science, in a way, so much more accessible and reliable, as well as of vastly greater use to the wider world.

2 Background and Related Work

As aforementioned, systems that leverage LLMs to automate the process of reproducing a paper’s code have been recently developed. Developed by researchers from the University of Science and Technology of China, Lin et al. (2025), AutoP2C [3] is the first of these systems. It introduces a multimodal, multi-agent pipeline that extracts information from text, figures, and tables and then iteratively debugs generated code. Its main strength is improving execution success through repeated refinement. However, AutoP2C focuses primarily on producing executable code rather than helping users understand how the implementation relates to the original paper.

Paper2Code [7], the second attempt at this process, was developed by researchers from the Korea Advanced Institute of Science and Technology, Seo et al. (2025), in collaboration with deepauto.ai. It takes a different approach than AutoP2C as it begins by generating a repository “blueprint” that outlines the expected structure of the codebase, then fills in the components using LLMs. While this top-down method provides cleaner organization, Paper2Code still evaluates success mainly through execution-based metrics such as structural completeness and replication score. Similar to AutoP2C, it does not provide traceability or any explanations that help users bridge the gap between textual descriptions and code.

While these systems are excellent and broke the ice on the concept of using LLMs for paper-to-code generation, they simply lack explainability, preventing them from being user-friendly. EP2C builds on this existing work and attempts to highlight missing details in the papers, clarify ambiguous implementation decisions, and show users how specific functions relate to the methods described in the text. While these systems are excellent and broke the ice on the concept of using LLMs for paper-to-code generation, they simply lack explainability, preventing them from being user-friendly. EP2C builds on this existing work and attempts to highlight missing details in the papers, clarify ambiguous implementation decisions, and show users how specific functions relate to the methods described in the text.

3 System Design

Within the EP2C pipeline, we start by checking a research-tracker¹ module from HuggingFace² to see whether the input paper already has an associated GitHub repository. If found, the repository is downloaded and used directly as the implementation. However, if there is no existing GitHub repository, the pipeline will begin by parsing the PDF into structured markdown using MinerU. Next, it will run three sequential phases adapted from the Paper2Code pipeline [7]: a planning phase that designs the repository structure and high-level architecture, an analysis phase that provides file-level logic and responsibilities and a coding phase that calls GPT based models to generate the actual code files and the explanation layer which we added on top of the pipeline. The pipeline writes out a complete repository together with markdown artifacts and returns a unified set of paths that the web app and PaperCodeSync viewer use to display side-by-side, traceable paper-to-code views.

3.1 Parsing

Information from the paper can only be obtained once it has been parsed. We evaluated existing methods in Paper2Code [7] and AutoP2C [3] to incorporate them into our data extraction and formatting.

The Paper2Code pipeline uses a system built on GROBID³ called S2ORC-Doc2JSON [4] that processes PDF files into XML and later JSON files. A processed paper’s resulting JSON file contains metadata, e.g. authors, publication dates, or titles, as well as the full textual content of the paper. Critically, this textual content does not include diagrams, graphs, and other non-text data in the paper. Image captions, figure titles, and some tables can be extracted, but never actual non-text data [1]. Our pipeline aimed to take advantage of modern LLMs capable of processing both text and image data, allowing us to reason over diagrams, figures, and other visual elements.

AutoP2C uses MinerU⁴ for its parsing step. This allows textual and image data to be incorporated into its pipeline phases directly [3]. MinerU generates, among other things, a markdown file containing text and image content in read-order [5]. AutoP2C uses this markdown file as input to its pipeline, hopefully allowing more context and nuance during planning, analysis, and code generation.

Our parsing step follows largely from AutoP2C with MinerU. We considered GROBID, PyPDF⁵, and even LLMs as alternatives, however none generated the output we desired as input for our pipeline. GROBID, as previously mentioned, does not handle non-textual data, thus omitting content from our pipeline and potentially the final repository. PyPDF can only extract embedded images, a trait that is not guaranteed for all PDF files, especially not those scanned physically [2]. Lastly, LLM-based PDF extraction did not seem to be a tested alternative to a known library like MinerU. While libraries do exist, they are not as comprehensive or deployable as MinerU [6].

Aside from its Mandarin documentation, MinerU allowed for straightforward implementation as a first step to parsing. In our pipeline, the given PDF is fed through MinerU, which outputs image files for diagrams, tables, and equations as well as JSON files for textual content. These JSON files contain an excess of information, like bounding boxes and title counters, meaning our second parsing step is to remove non-required content before passing the PDF data to an LLM. So, we compile text and images in *read-order* from the MinerU output files to feed into our pipeline. This way, LLMs receive only the informative content of the paper, from the standpoint of reproducing its methods. Note that our pipeline uses MinerU’s standard “pipeline” mode. We attempted to use their VLM-transformer mode, but this would have significantly slowed down implementation. It should also be noted that MinerU’s markdown file output, the same one used in AutoP2C, is used for our UI Paper Code Sync layer.

3.2 Planning Phase

In this phase, we built a multi-stage driver that takes the parsed research paper and turns it into a structured blueprint for code generation. The script first loads all the environment configuration `.env`, parses all the CLI arguments such as `paper_name`, `gpt_version`, and then ingest the paper. It also combines the text and figure and encodes the images to base64 so they can be passed to the OpenAI model.

Apart from this input layer, the pipeline runs four consecutive chat stages with the API: an overall reproduction plan, an architecture and file list, a task and dependency breakdown, and finally a paper grounded `config.yaml`. The planning phases concludes by extracting the YAML config into `planning_config.yaml` and generating a human readable `PLANNING.md` that consolidates all the planning outputs for review and for subsequent analysis phase.

3.3 Analysis Phase

In the analysis phase, we take the outputs of the planning step and turn them into file level guidance to prepare the ground for code generation. The driver then loads the paper, the `planning_config.yaml` from the planning phase and the planning conversation from `planning_trajectories.json`, then recovers the high-level design and task description. It obtains the canonical task list and logic descriptions and build a mapping from each target file to its associated “Logic Analysis” description.

For every file in the task list(excluding `config.yaml`),the script creates a prompt that includes the paper content, the plan, the design JSON, the task specification and the full configuration and sends this along with a system messages that instructs the model to follow the existing data structures. The responses are logged with cost-tracking, saved as per-file JSON dumps, and written as human-readable analysis notes under `analyzing_artifacts`.Finally, it combines all the notes into `ANALYSIS.md`, providing a structured, file-by-file description of responsibilities and dependencies that guides the subsequent coding phase.

3.4 Coding Phase

During the coding phase of the pipeline, EP2C generates actual code files from the outputs of the planning and analysis stages. The `3_coding.py` script is responsible for this whole process as it begins by loading the parsed paper (in JSON or markdown), the `planning_config.yaml` file and the conversation stored in `planning_trajectories.json`. The script then reads the earlier saved task list and per-file analysis results which were stored as `*_simple_analysis_response.json` files. These inputs provide the script with a clear list of files to generate along with a detailed description of each file’s function and what order the files are generated in. For each file in the task list (except for `config.yaml`) the script creates a prompt that contains the paper content, the high-level plan, the design JSON, the file’s task specification, the full configuration

and any code that has already been generated for earlier files. This prompt is sent to a GPT-based model with a fixed system message that emphasizes following the planned structure, utilizing the correct types and default values and not making any changes to the existing data structures that has been selected. The model's output is recorded, we monitor the cost as well and the raw output is stored as a coding artifact in the `coding_artifacts` folder. The script then takes the code blocks from the responses and places it in the generated repository at the designated file path. The code that is produced is also kept in memory so that it can be offered as context when generating other files. Once all files are processed, the pipeline produces a `CODING.md` file that gathers all the model outputs for each file giving the frontend a way to display how the final codebase was developed.

3.5 Explanation Phase

At the end of coding phase the explanation phase starts and the explanation layer runs on top of the generated repository. First the script gets a JSON version of the paper: if `paper_content.json` is already there in the output folder it will be utilized if not, it tries to use the original JSON input or it transforms the markdown into a simple JSON structure and if nothing works it creates a small fallback JSON file. Inside this pipeline, EP2C loads the paper JSON, all the generated code files, the planning outputs, the config and first builds a traceability map that links paper sections to code pieces and also code back to the paper. It then runs a missing information detector to spot things like hyperparameters, datasets or hardware details that show up in the code or config but are not clearly described in the paper. After that, it writes a `README.md` and a full `EXPLANATION.md` that describe the project, the code structure, the traceability links and the missing information in normal language that a user can read. The explanation layer also calculates some explainability metrics to see how well the code can be understood and linked back to the paper:

- Traceability coverage tells us how much of the paper is covered by code by counting how many paper sections have code that connects to them, divided by the total number of sections.
- Comment density calculates how well-documented the code is by counting comment lines (lines starting with `#` or containing docstrings) and dividing by the total number of lines.
- Paper reference accuracy checks for how often the code mentions the paper by looking for phrases like "section 3", "equation (1)" or "figure 2" and calculates what percentage of files contain these references.
- Missing information score looks for how complete the information is by giving points for missing items based on how serious they are (high severity items get 3 points, medium get 2, low get 1) then inverting the score so fewer missing items get a higher score.
- Readability score tells us how easy the code is to understand by checking if files have docstrings, if functions have documentation, if type hints are used and how many inline comments there are then averaging these scores across all files.

All 5 metrics are combined into one overall score using weighted averages and traceability coverage weighted most heavily at 30% to give a single number from 0% to 100% showing how explainable the generated code is. All of the outputs live in the `explanation_layer` folder and are used by the UI to help users see not just that the code is generated but where it comes from in the paper and how well did it get generated.

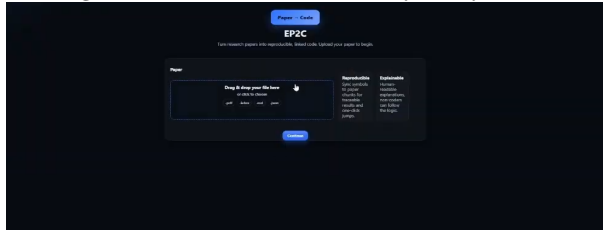
3.6 Web App

Our web application provides an intuitive, user-friendly interface for uploading academic papers. After a user submits a paper, the EP2C pipeline processes the content and generates an associated code repository. Once the code is ready, users can explore the repository, view corresponding explanations, try the links between code and paper sections, and download the entire project as a ZIP file. This seamless interaction between paper and code is designed to improve the reproducibility of research and enhance the transparency of generated code.

3.7 Paper Code Sync Sub-Pipeline

In order to enhance the reproducibility and transparency of research papers, we really wanted to establish a seamless link between the papers and the code our EP2C pipeline generated. A huge source of inspiration

Figure 1: *Frontend User-Interface of EP2C*



for our approach came from Overleaf⁶. They have this code jump to view feature, which they call SyncTeX⁷, and it works exactly how we want our version to do. They create a mapping between the sections of a LaTeX document and the corresponding output to create a mapping. Overleaf focuses on mapping text to visual outputs, but our challenge was actually synchronizing the code with the paper itself. This was far more complicated than originally anticipated due to the separate nature of code and paper artifacts. That said, we still adopted the underlying concept of creating a mapping from SyncTeX.

The first module in our papercodesync sub-pipeline focuses on breaking down the paper’s content into literal chunks of text. Attempts were made to parse the paper using GROBID³, and although successful at first, ultimately it was too time-consuming and given our tight time constraint, we decided to simply reuse our MinerU output from the actual EP2C pipeline. Now, using the markdown output that MinerU gives us, we use it as a starting point from which we normalize any LaTeX, equations, etc., and split it into sections. Long sections are also divided into smaller parts to increase digestibility, and each chunk is assigned a hash for future identification. This segmentation of the paper into chunks is essential for the later stages where each chunk will be linked with an actual piece of code.

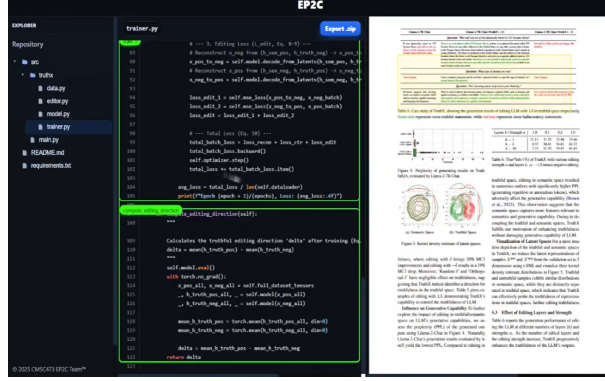
The second module deals with literally extracting sections from our code repository, which we call symbols. A symbol is essentially a function, or class, or span of code, etc. We implement a Tree-sitter⁸, a powerful incremental parser, to actually analyze our code and capture the syntax of the various programming languages that the code repositories may have. This module supports multiple languages, such as Python, JavaScript, and Java, just to name a few. Each symbol is further annotated with additional metadata, such as line numbers, signature, docstring, etc. These symbols are also hashed with unique IDs.

The third and last module in this sub-pipeline focuses on actually creating a map that links the code symbols to the actual paper chunks. We start by creating an overlap index (we use both BM25⁹ and TF-IDF¹⁰, which the user can pick in the config) to find literal text matches. Then, we build a separate semantic index using sentence-transformers¹¹ to find meaning-based matches. Next, we build a query from the symbols (bag of words¹²) and use it on both indexes to find the top k matches. Afterwards, we’ll normalize the scores (using min-max¹³) and then line up every chunk that appeared in both lists. Finally, we compute a weighted blend by combining the scores with a configurable weight to sort with. The best match from here is what we use in our actual link. By no means is this perfect, and it needs a lot of tuning, but it certainly does assist the user in quickly identifying which part of the code corresponds to each section of the paper, ultimately improving our transparency and reproducibility of the research process, which is what we were aiming to do.

4 Example

We have provided a video demonstration of the full EP2C pipeline from start to finish. In the video, the user clones our GitHub repository, follows the **Usage** instructions, and generates a repository. Note that setting up the environment is cut out of the video, as it would show personal API keys. The video shows both the web app and the terminal pipeline output so that each step is clear. The first step involves parsing the PDF file with MinerU. Next, the four phases play out, and finally the UI synchronization occurs. Then the web app updates with the generated repository, and the user starts reading code next to the original paper.

Figure 2: User Clicking a Code Symbol with the Paper Highlighting the Matching Section



5 Evaluation and Performance

Our evaluation metrics follow from Paper2Code’s agent-based evaluation system. We adapt their reference-free evaluation implementation to fit our four phase pipeline. This method allows an AI agent to evaluate the completeness and accuracy of the generated repository and give a numeric score averaged over eight samples. However, our scores range from 1.00 to 5.00, while Paper2Code’s scores range only from integer values 1 to 5 [7].

So, we use reference-free evaluation on 42 of the 90 papers from the Paper2Code benchmark dataset¹⁴.

Table 1: A small sample of the comparison of Paper2Code and EP2C scores with global mean values.

Paper	Paper2Code	EP2C	EP2C – P2C
llm-detector-evasion	2.10	2.45	+ 0.35
ACT	2.15	3.05	+ 1.10
llm-pddl-planning	2.45	2.10	– 0.35
LPFIInISR	3.45	4.05	+ 0.60
mar	2.45	2.45	+ 0.00
G-Retriever	3.10	3.55	+ 0.45
Mean	~ 2.48	~ 2.82	

Note that both repository generation and agent-based evaluation were performed, for affordability, with GPT-4.1-nano¹⁵. This results in values much further from 5.00 than if a flagship model was used. Also note that the mean values in the table above reflect the scores of all 42 papers tested, not just scores from the sample shown.

So, using the Paper2Code benchmark dataset, we have a sample mean $\bar{x} \approx 2.82$ and thus a sample std.

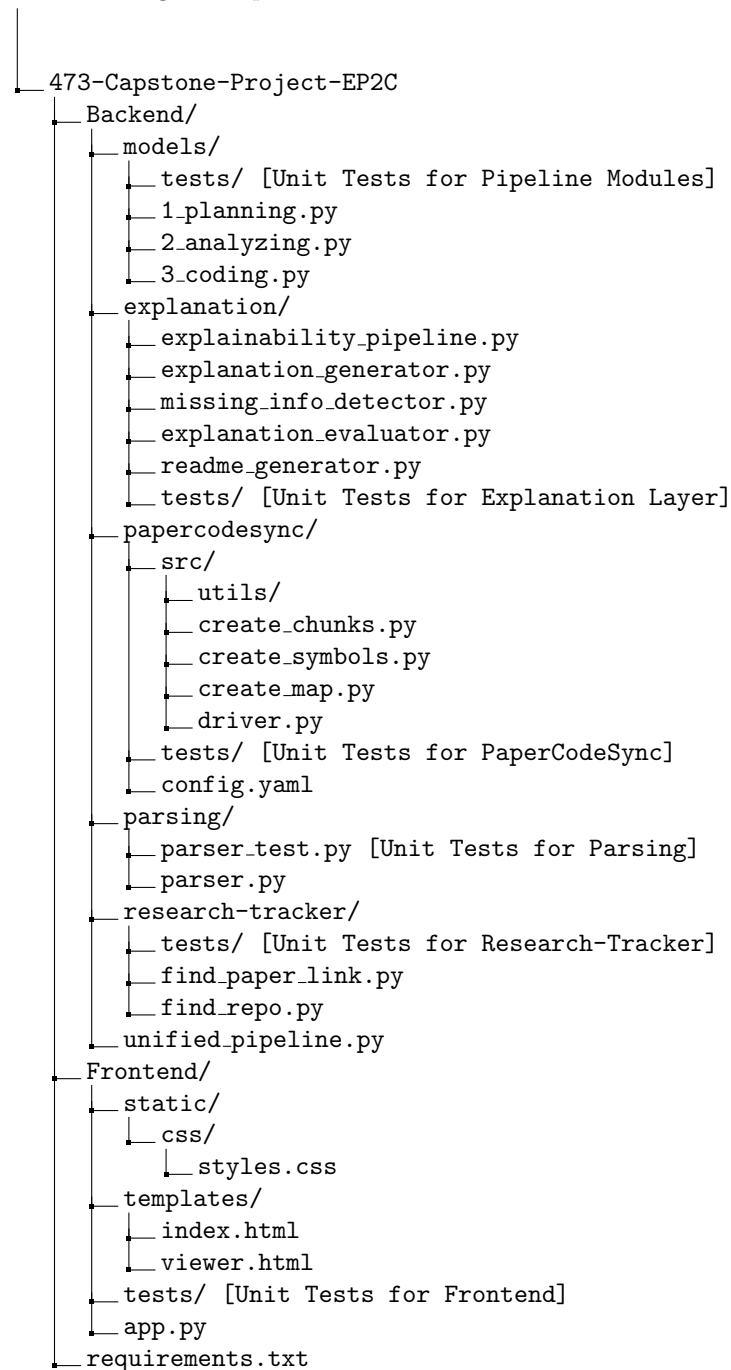
of $\sigma \approx \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2} \approx 0.67$. Considering a null hypothesis $H_0 : \bar{x} = \mu_0$, where the Paper2Code mean

is $\mu_0 = 2.48$, we can set our hypothesis $H : \bar{x} > \mu_0$. For a t -distribution with 41 degrees of freedom and $\alpha = 0.05$, this results in a one-tailed critical value of $t_c \approx 1.68$. Given our data, we have $t \approx 3.26 \gg t_c$. Thus, we reject the null hypothesis, and conclude that EP2C outperforms Paper2Code on average.

This improvement in performance can be attributed to two major changes made by our pipeline. Our integrated parsing step removes significantly more bloat as well as captures non-textual data that GROBID misses in Paper2Code’s parsing step. This improvement carries throughout the pipeline, as more precise and complete input directly changes outputs of each phase. The other significant change that is likely impacting performance is the addition of our Explainability Layer. Agent-based evaluation likely benefits from more documentation existing in a repository, as this contributes to repository completeness.

6 Important Files

The following are important files and unit tests within our GitHub repository.



7 Reflection on Goals

In our proposal, EP2C's main goal was a generating both a runnable repository with the added explanation layer, since previous paper-to-code system usually try to make sure the code run and do not try to connect implementations back to the paper. In our final system, we delivered that through a structured pipeline: parsing papers into consistent markdown with MinerU, using the HuggingFace research tracker to find the relevant datasets and code, and then generating repositories via stages: planning, analysis, code generation,

all while keeping the configuration grounded in what the paper actually states.

We also delivered the proposal’s main differentiator: traceability by adding an explanation phase that maps the code components to the relevant paper section, flags missing details like hyperparameter/dataset and generates a human readable README documentation. For evaluation, we followed through on benchmarking against Paper2Code by adapting its agent based scoring and reported a higher mean score for EP2C on the sample (2.82 in contrast to Paper2Code’s 2.48). One planned area we adjusted was finetuning code generation models: we switched to prompt-based API calls because fine-tuning was too time consuming, required large memory and unreliable for our constraints.

8 Discussion and Future Work

In the future, there are so many ways we can improve this existing EP2C pipeline. One way is to use the transformer version of MinerU so the system can handle more complicated page layouts and read figures and tables way more accurately. We also want to fine tune on bigger LLM models with data that is closer to this task like pairs of research papers and their code instead of only using general models (we tried this on a small scale model and turned out to be very time consuming, error-prone and not effective). Another important area is our explanation and evaluation metrics. Right now we look at things like coverage and comments in Python but we still miss some topics and gaps in the paper so we want to design better metrics that catch more missing details and measure how clear and honest the explanations really are. Finally, we plan to tune the hyperparameters in the PaperCodeSync pipeline such as how we chunk the paper and how strict the matching rules are so we can get closer and more reliable links between what is written in the paper and the code that EP2C generates.

9 Conclusions

EP2C tackled a number of challenges in bridging academic research with practical implementation, and through careful system design, we delivered a functional end-to-end prototype. While existing tools focus primarily on producing runnable code, EP2C demonstrates that explainability and traceability can be meaningfully integrated into the paper-to-code process. Our work shows that a more transparent and user-guided approach to code generation is both feasible and valuable, laying a foundation for more accessible and reproducible machine learning research.

10 Acknowledgments

We would like to thank the CMSC473 teaching staff for their guidance, as well as the authors of the AutoP2C (Lin et al., 2025) and Paper2Code (Seo et al., 2025), whose framework formed the backbone of our implementation.

References

- [1] Grobid. <https://github.com/kermitt2/grobid>, 2008–2025.
- [2] FENNIAC, M., STAMY, M., PUBPUB ZZ, THOMA, M., PEVELER, M., EXILEDKINGCC, AND PYPDF CONTRIBUTORS. The pypdf library, 2024. See <https://pypdf.readthedocs.io/en/latest/meta/CONTRIBUTORS.html> for all contributors.
- [3] LIN, Z., SHEN, Y., CAI, Q., SUN, H., ZHOU, J., AND XIAO, M. Autop2c: An llm-based agent framework for code repository generation from multimodal content in academic papers, 2025.
- [4] LO, K., WANG, L. L., NEUMANN, M., KINNEY, R., AND WELD, D. S2ORC: The semantic scholar open research corpus. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (Online, July 2020), Association for Computational Linguistics, pp. 4969–4983.
- [5] NIU, J., LIU, Z., GU, Z., WANG, B., OUYANG, L., ZHAO, Z., CHU, T., HE, T., WU, F., ZHANG, Q., JIN, Z., LIANG, G., ZHANG, R., ZHANG, W., QU, Y., REN, Z., SUN, Y., ZHENG, Y., MA, D., TANG, Z., NIU, B., MIAO, Z., DONG, H., QIAN, S., ZHANG, J., CHEN, J., WANG, F., ZHAO, X., WEI, L., LI, W., WANG, S., XU, R., CAO, Y., CHEN, L., WU, Q., GU, H., LU, L., WANG, K., LIN, D., SHEN, G., ZHOU, X., ZHANG, L., ZANG, Y., DONG, X., WANG, J., ZHANG, B., BAI, L., CHU, P., LI, W., WU, J., WU, L., LI, Z., WANG, G., TU, Z., XU, C., CHEN, K., QIAO, Y., ZHOU, B., LIN, D., ZHANG, W., AND HE, C. Mineru2.5: A decoupled vision-language model for efficient high-resolution document parsing, 2025.

- [6] RUSLANMV. Extracting data from pdfs with local llm. GitHub repository, 2025. Accessed: 2025-12-17.
- [7] SEO, M., BAEK, J., LEE, S., AND HWANG, S. J. Paper2code: Automating code generation from scientific papers in machine learning, 2025.

Notes

¹The Research Tracker is a Hugging Face Space created by dylanebert (Dylan Ebert) that serves as an interface for an AI research intelligence tool. It helps users discover and organize information related to AI/ML research, such as papers, code, and models, by automating the process of cross-referencing information across multiple platforms.

²Hugging Face is a leading AI platform and community, often called the "GitHub for Machine Learning," providing a central hub for developers to find, share, and collaborate on open-source models, datasets, and AI applications, especially for Natural Language Processing (NLP).

³GROBID (GeneRation Of Bibliographic Data) is an open-source machine-learning tool for extracting metadata, bibliographic references, and full-text content from PDF files.

⁴MinerU is an open-source document parsing framework developed by OpenDataLab that converts scientific papers into structured, machine-readable formats, including Markdown and JSON. It preserves the hierarchical structure of the document, such as sections, lists, equations, and figures, making it particularly suitable for downstream tasks like code generation and algorithm extraction.

⁵PyPDF is a Python library used for working with PDF files programmatically. It enables users to read and extract text from PDFs, merge and split documents, rotate and crop pages, and add or modify metadata, making it a practical tool for basic PDF manipulation and automation tasks.

⁶Overleaf is a popular, collaborative, online LaTeX editor that simplifies writing and typesetting complex scientific, technical, and academic documents without needing local LaTeX installation.

⁷SyncTeX (Synchronize TeXnology) is a powerful tool for LaTeX and TeX users that links your source code (.tex file) directly to its output (PDF), allowing you to click in one to jump to the corresponding location in the other, streamlining debugging and navigation in large documents.

⁸Tree-sitter is a powerful parsing library and parser generator designed to efficiently create and manage concrete syntax trees (CSTs) from source code in various programming languages.

⁹BM25, or Best Matching 25, is a popular ranking function used in information retrieval and search engines to determine the relevance of documents to a user's search query.

¹⁰TF-IDF (Term Frequency-Inverse Document Frequency) in an index is a numerical weighting scheme that scores word importance in a document relative to a whole collection (corpus).

¹¹Sentence Transformers are a type of neural network model that converts text into dense vector representations, or "embeddings". These embeddings are numerical vectors where sentences with similar meanings are close together in the vector space, making it possible to perform tasks like semantic search, clustering, and similarity analysis.

¹²A Bag-of-Words (BoW) model is a simple text representation method in NLP that treats text as an unordered collection (a "bag") of its words, ignoring grammar and word order but counting word frequencies to create numerical vectors for documents, useful for tasks like spam filtering or sentiment analysis.

¹³Min-max normalization is a data scaling technique that transforms features into a specific range, most commonly $[0, 1]$.

¹⁴The Paper2Code benchmark is a collection of 90 papers from 3 different conferences. This dataset and its usage can be found on their GitHub in the `data/paper2code/` directory.

¹⁵GPT-4.1-nano is designed to be smaller and more cost-efficient than larger GPT-4-class models by prioritizing lightweight architecture and reduced computational overhead. It uses fewer parameters and streamlined inference pathways, which lower memory usage and latency while still preserving strong general-purpose language understanding. These design choices make its use through an API much cheaper.